

Query params en Angular



Concepto de queryParams en Angular

En Angular, los queryParams (o parámetros de consulta) son una forma de pasar información adicional en la URL de una aplicación sin cambiar la ruta principal. Los parámetros de consulta aparecen en la URL después de un signo de interrogación (?) y se utilizan comúnmente para filtrar datos, paginación, o para mantener el estado de una vista.

Los queryParams son opcionales y no afectan la ruta principal. Puedes agregar varios queryParams a una URL, y cada parámetro está separado por un símbolo &.

Sintaxis de queryParams

Cuando trabajas con queryParams en Angular, normalmente los utilizas en conjunto con el Router para navegar entre componentes y para manejar las rutas. Los queryParams se pueden añadir usando la propiedad [queryParams] en el enlace de una ruta.

Sintaxis básica:

```
this.router.navigate(['/ruta'], { queryParams: { parametro: 'valor' } });
```

O en la plantilla HTML:

```
<a [routerLink]="['/ruta']" [queryParams]="{ parametro: 'valor' }">Enlace</a>
```

Ejemplo básico en Angular

Supongamos que tienes una ruta a un componente llamado `PersonasComponent`, y quieres pasar un parámetro de consulta llamado `filtro` para filtrar una lista de personas.

1. Definir la navegación con `queryParams`:

En el componente que navega:

```
import { Component } from '@angular/core';
import { Router } from '@angular/router';

@Component({
  selector: 'app-home',
  template: `
    <button (click)="filtrarPersonas('activos')">Ver Activos</button>
  `
})
export class HomeComponent {
  constructor(private router: Router) {}

  filtrarPersonas(filtro: string) {
    this.router.navigate(['/personas'], { queryParams: { filtro: filtro } });
  }
}
```

En la plantilla:

```
<button (click)="filtrarPersonas('activos')">Ver Activos</button>
```

2. Recibir y usar `queryParams` en el componente de destino (`PersonasComponent`):

```
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';

@Component({
  selector: 'app-personas',
  template: `
    <div *ngIf="filtro">
      Mostrando personas con el filtro: {{ filtro }}
    </div>
    <div *ngIf="!filtro">
      Mostrando todas las personas.
    </div>
  `
})
export class PersonasComponent implements OnInit {
  filtro: string | null = null;

  constructor(private route: ActivatedRoute) {}

  ngOnInit() {
    this.route.queryParams.subscribe(params => {
      this.filtro = params['filtro'] || null;
    });
  }
}
```

3. Estructura de URL con `queryParams`:

Cuando haces clic en el botón, la URL se verá algo como:

<http://www.globalmentoring.com.mx>

`http://localhost:4200/personas?filtro=activos`

En el `PersonasComponent`, podrás usar el valor del filtro para filtrar la lista de personas o realizar otras operaciones.

Pasar Múltiples queryParams en Angular

Si deseas pasar más de un parámetro de consulta, puedes incluirlos en un objeto dentro de la propiedad `queryParams`. Cada clave en el objeto representa un parámetro de consulta diferente.

Ejemplo Completo

1. Navegar con múltiples queryParams

Supongamos que deseas pasar dos parámetros, filtro y orden, al componente de destino.

En el componente que navega (`HomeController`):

```
import { Component } from '@angular/core';
import { Router } from '@angular/router';

@Component({
  selector: 'app-home',
  template: `
    <button (click)="filtrarPersonas('activos', 'asc')">
      Ver Activos en Orden Ascendente</button>
    <button (click)="filtrarPersonas('inactivos', 'desc')">
      Ver Inactivos en Orden Descendente</button>
  `
})
export class HomeComponent {
  constructor(private router: Router) {}

  filtrarPersonas(filtro: string, orden: string) {
    this.router.navigate(['/personas'], {
      queryParams: { filtro: filtro, orden: orden }
    });
  }
}
```

En la plantilla:

```
<button (click)="filtrarPersonas('activos', 'asc')">Ver Activos en Orden Ascendente</button>
<button (click)="filtrarPersonas('inactivos', 'desc')">Ver Inactivos en Orden
Descendente</button>
```

2. Recibir y utilizar múltiples queryParams en el componente de destino (`PersonasComponent`):

En el componente `PersonasComponent`:

```
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';

@Component({
```

```

    selector: 'app-personas',
    template: `
      <div *ngIf="filtro && orden">
        Mostrando personas con el filtro: {{ filtro }} en orden: {{ orden }}
      </div>
      <div *ngIf="!filtro && !orden">
        Mostrando todas las personas.
      </div>
    `
  })
}
export class PersonasComponent implements OnInit {
  filtro: string | null = null;
  orden: string | null = null;

  constructor(private route: ActivatedRoute) {}

  ngOnInit() {
    this.route.queryParams.subscribe(params => {
      this.filtro = params['filtro'] || null;
      this.orden = params['orden'] || null;
    });
  }
}

```

3. Estructura de URL con múltiples queryParams:

Cuando haces clic en uno de los botones, la URL podría verse así:

`http://localhost:4200/personas?filtro=activos&orden=asc`

Esto significa que estás pasando dos parámetros de consulta: filtro con el valor activos y orden con el valor asc.

Resumen:

- **Pasar múltiples queryParams:** Puedes pasar tantos queryParams como necesites, simplemente agregándolos al objeto queryParams.

```

this.router.navigate(['/ruta'], { queryParams: { param1: 'valor1', param2: 'valor2', param3: 'valor3' } });

```

- **Recuperar múltiples queryParams:** En el componente de destino, puedes acceder a ellos utilizando ActivatedRoute y suscribiéndote a queryParams.

```

this.route.queryParams.subscribe(params => {
  this.param1 = params['param1'];
  this.param2 = params['param2'];
  // ...
});

```

Esta técnica es útil para pasar múltiples parámetros de consulta para controlar aspectos como filtros, ordenación, paginación, o cualquier otro dato que necesites manejar en tu aplicación Angular.

En Angular puedes usar el método snapshot del ActivatedRoute para acceder a los queryParams sin necesidad de suscribirte. Ambas técnicas (snapshot y subscribe) tienen sus casos de uso específicos y consideraciones en cuanto a cuál es la mejor práctica.

Usando snapshot

El snapshot es una instantánea del estado actual de la ruta, incluyendo los parámetros de consulta. Es útil si sabes que los parámetros no cambiarán mientras el usuario esté en la misma instancia del componente.

Ejemplo con snapshot:

```
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';

@Component({
  selector: 'app-personas',
  template: `
    <div *ngIf="filtro && orden">
      Mostrando personas con el filtro: {{ filtro }} en orden: {{ orden }}
    </div>
    <div *ngIf="!filtro && !orden">
      Mostrando todas las personas.
    </div>
  `
})
export class PersonasComponent implements OnInit {
  filtro: string | null = null;
  orden: string | null = null;

  constructor(private route: ActivatedRoute) {}

  ngOnInit() {
    this.filtro = this.route.snapshot.queryParams['filtro'] || null;
    this.orden = this.route.snapshot.queryParams['orden'] || null;
  }
}
```

snapshot vs. subscribe

1. snapshot:

- **Uso:** Se utiliza cuando no esperas que los parámetros cambien mientras el usuario está en la misma instancia del componente.
- **Ventajas:**
 - Es simple y directo.
 - No necesitas gestionar la suscripción ni desuscribirte (evita posibles fugas de memoria).
- **Limitaciones:**
 - No reaccionará a cambios en los parámetros de consulta si estos cambian mientras el componente está activo.

2. subscribe:

- **Uso:** Es útil cuando los queryParams pueden cambiar mientras el usuario navega dentro del mismo componente (por ejemplo, si hay botones o enlaces que cambian los parámetros de consulta sin recargar el componente).
- **Ventajas:**
 - El componente reaccionará a cualquier cambio en los queryParams mientras esté activo.
- **Limitaciones:**
 - Requiere suscribirse y, en algunos casos, desuscribirse para evitar fugas de memoria, aunque Angular maneja esto bien con suscripciones a ActivatedRoute que se limpian cuando el componente se destruye.

¿Cuál es mejor práctica?

- **Usa snapshot** si los parámetros de consulta no van a cambiar mientras el componente está activo. Esto es más eficiente y simple cuando no necesitas responder a cambios dinámicos en los queryParams.
- **Usa subscribe** si necesitas que tu componente reaccione a cambios en los queryParams mientras está activo. Por ejemplo, si tienes un componente de lista que filtra datos según los parámetros de consulta, y el usuario puede cambiar esos parámetros sin recargar la página.

En general, elige la técnica que mejor se adapte a las necesidades de tu componente:

- **Estático:** snapshot
- **Dinámico:** subscribe

Saludos!

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](http://www.globalmentoring.com.mx)